

AADK 5 - Task 3 – Investigate State, Immutability & Side Effects in Compose Contexts

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 5

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

Research

Definitions

Immutable Object

An immutable object is one whose state cannot be changed after it is created. In Kotlin, this is typically achieved using val properties and avoiding mutable collections. Any “change” requires creating a new object.

Mutable State

Mutable state refers to data that can change over time. In Compose, mutable state (e.g., mutableStateOf) is observable and triggers recomposition when updated.

Recomposition

Recomposition is the process where Jetpack Compose re-executes composable functions when observed state changes, updating only the necessary UI parts.

Side Effect

A side effect is any change that occurs outside the scope of a function’s return value (e.g., modifying global variables, writing to disk, logging, launching coroutines).

Real-World Examples of Mutable State Bugs

1. **RecyclerView List Not Updating (Android View System)**

Developers modified an existing ArrayList but forgot to call notifyDataSetChanged(). UI did not update even though data changed.

2. Compose List Not Recomputing

A `mutableListOf()` was modified directly (e.g., `list.add()`), but because Compose wasn't observing it as state, recomposition did not occur.

2 Experiment

✗ Example 1: Mutable List (UI Might Not Update)

```
val taskList = mutableListOf("Task 1", "Task 2")
```

```
fun addTask(task: String) {  
    taskList.add(task) // Mutates existing list  
}
```

⚠ Conceptual Issue in Compose:

If this list is not wrapped in `mutableStateOf` or `mutableStateListOf`, Compose will not detect changes.

Even if it is state-backed, directly mutating the list may not trigger recomposition reliably.

Expected: UI updates automatically.

What Might Go Wrong: UI remains unchanged because state observer wasn't triggered.

✓ Example 2: Immutable List + Copy Approach (Safer)

```
var taskList by mutableStateOf(listOf("Task 1", "Task 2"))
```

```
fun addTask(task: String) {  
    taskList = taskList + task // Creates new list  
}
```

✓ Why This Works:

- `taskList` reference changes.
- Compose detects state update.
- Recomposition occurs automatically.

Expected: UI updates correctly.

Why Safer: No shared mutable structure; state transitions are predictable.

✅ Using copy() with Data Class

```
data class Task(val id: Int, val title: String, val isCompleted: Boolean)
```

```
fun toggleTask(task: Task): Task {  
    return task.copy(isCompleted = !task.isCompleted)  
}
```

Instead of mutating task.isCompleted, we create a new object.

State Management Plan (For Task Tracker Feature from Task 2)

Feature: **Task Tracker**

📌 State Plan:

- Store tasks as `MutableState<List<Task>>` or `StateFlow<List<Task>>` inside `ViewModel`.
- When user toggles completion → create a new Task using `copy()` → update list immutably.
- Never modify `mutableListOf()` directly inside UI layer.
- Keep repository logic separate from composables.
- Use `remember` or `collectAsState()` to observe state changes in Compose.

🚫 Avoiding Side Effects:

- No global mutable variables.
- No direct mutation inside composables.
- Use `LaunchedEffect` only for controlled side-effects.
- Keep state updates inside `ViewModel`.
- Use unidirectional data flow (State → UI → Event → State update).

Reflection

Immutability is crucial for reliable UI apps because it ensures predictable state transitions. When objects don't change internally, debugging becomes easier and unintended side effects are minimized. In Compose, recomposition depends on detecting state changes — and immutable updates guarantee a new reference, making recomposition reliable. Kotlin's data class and `copy()` function make immutability practical and expressive, supporting clean and maintainable UI architecture.